

Revisão de Estrutura de Árvores

Nome: Fellipe Gabriel Souza de Assis Monteiro

Disciplina: Estrutura de Dados II

Professora: Profa. Kadidja Valéria

Data: 28/09/2026

Etapa 1 - Revisão bibliográfica

1. Conceitos Básicos de Árvore

- **Nó:** Unidade básica que compõe a árvore, contendo dados e ponteiros (ou referências) para outros nós.
 - **Raiz:** O nó principal, localizado no topo da hierarquia, que não possui nenhum nó pai.
 - **Pai e Filho:** Relação hierárquica em que um nó aponta para outro diretamente abaixo dele (filho) ou acima (pai).
 - **Folha:** Nó terminal que não possui nenhum filho.
 - **Altura:** O comprimento do caminho mais longo da raiz até uma folha (ou o número de arestas desse caminho).
 - **Percurso:** Estratégias de visitação sistemática de todos os nós da árvore (ex: pré-ordem, em ordem, pós-ordem e em largura).
-

2. Árvore Geral (General Tree)

- **Organização dos dados:** Os nós podem ter um número arbitrário de filhos, sem limite fixo por nó.
 - **Propriedade mantida:** Relação estrita de hierarquia n-ária (um nó pode gerenciar múltiplos sub-ramos).
 - **Busca e Inserção:** A busca é feita por varredura ou percursos (como largura/profundidade), uma vez que não há ordenação restrita padrão. A inserção ocorre vinculando o novo nó como filho de um nó pai designado.
 - **Ajustes:** Geralmente não possui auto-balanceamento nativo, exigindo reestruturações manuais dependendo da aplicação.
 - **Contexto de utilidade:** Representação de hierarquias corporativas, sistemas de arquivos (pastas e subpastas) e estruturas de documentos (como XML ou HTML).
-

3. Árvore Binária (Binary Tree)

- **Organização dos dados:** Cada nó possui no máximo dois filhos, convencionalmente denominados filho esquerdo e filho direito.
 - **Propriedade mantida:** Grau máximo de ramificação igual a dois.
 - **Busca e Inserção:** A busca requer o exame de subárvores esquerda e direita se não houver ordenação, resultando em complexidade $O(N)$ no pior caso. A inserção busca o primeiro espaço livre conforme a política da estrutura (ex: árvore binária cheia ou completa).
 - **Ajustes:** Nenhuma propriedade de balanceamento é garantida por padrão, podendo degenerar para uma estrutura linear (lista encadeada).
 - **Contexto de utilidade:** Expressões aritméticas (árvores de expressão) e algoritmos de decisão (árvores sintáticas de compiladores).
-

4. Árvore Binária de Busca (ABB / BST)

- **Organização dos dados:** Valores ordenados em nós binários.
 - **Propriedade mantida:** Para qualquer nó, todos os valores na subárvore esquerda são menores que o valor do nó, e todos os valores na subárvore direita são maiores.
 - **Busca e Inserção:** A busca compara a chave procurada com o nó atual, indo para a esquerda se for menor ou para a direita se for maior ($O(\log N)$ no caso médio). A inserção segue o mesmo caminho de busca até encontrar a posição nula correta.
 - **Ajustes:** Se inserções ocorrerem de forma ordenada, a árvore pode ficar desbalanceada (desperdçando a eficiência para $O(N)$).
 - **Contexto de utilidade:** Tabelas de símbolos dinâmicas e conjuntos ordenados onde operações de busca rápida são necessárias em cenários de dados semi-aleatórios.
-

5. Árvore AVL

- **Organização dos dados:** Árvore binária de busca auto-balanceada.
- **Propriedade mantida:** Para todo nó, a diferença de altura entre a subárvore esquerda e a direita (fator de balanceamento) deve ser estritamente -1, 0 ou 1.
- **Busca e Inserção:** A busca funciona exatamente como em uma ABB. A inserção é feita como na ABB, mas seguida de uma verificação de balanceamento de baixo para cima ao longo do caminho de inserção.
- **Ajustes:** Caso o fator de balanceamento viole a regra (>1 ou <-1), são aplicadas rotações simples ou duplas (LL, RR, LR, RL) para restaurar o equilíbrio.

- **Contexto de utilidade:** Aplicações que exigem buscas frequentes e rigorosamente rápidas com menor tolerância a degradação de desempenho (ex: sistemas de memória e bases de dados em memória).
-

6. Árvore Rubro-Negra (Red-Black Tree)

- **Organização dos dados:** Árvore binária de busca onde cada nó armazena um bit adicional de cor (rubro/vermelho ou negro/preto).
 - **Propriedade mantida:** 1) A raiz é preta; 2) Folhas (nós nulos/NIL) são pretas; 3) Se um nó é vermelho, seus filhos são pretos; 4) Todo caminho de um nó até suas folhas descendentemente possui o mesmo número de nós pretos (altura negra).
 - **Busca e Inserção:** Busca idêntica à ABB. A inserção insere o nó como vermelho e valida as regras de cores.
 - **Ajustes:** Recolorações e rotações são aplicadas para corrigir violações de propriedades, garantindo que o caminho mais longo não seja mais do que o dobro do caminho mais curto.
 - **Contexto de utilidade:** Bibliotecas padrão de linguagens (como `std::map` e `std::set` no C++, ou `TreeMap/TreeSet` em Java) devido ao bom equilíbrio entre custo de inserção/remoção e busca.
-

7. Árvore B (B-Tree)

- **Organização dos dados:** Árvore de busca balanceada em blocos, amplamente ramificada (multivias), onde cada nó pode conter múltiplas chaves e múltiplos ponteiros para filhos.
 - **Propriedade mantida:** Todos os nós folha estão no mesmo nível. Os nós possuem limites mínimo e máximo de chaves regulados por uma ordem m .
 - **Busca e Inserção:** A busca percorre as chaves ordenadas dentro de cada nó para decidir qual ramificação seguir. A inserção ocorre nas folhas; se um nó transborda (ultrapassa a capacidade máxima), ele sofre uma divisão (*split*) e o elemento mediano sobe para o pai.
 - **Ajustes:** Divisões e fusões de nós para manter o balanceamento perfeitamente rígido nas pontas.
 - **Contexto de utilidade:** Sistemas de arquivos e bancos de dados relacionais baseados em disco rígido ou SSD, onde minimizar acessos de I/O de bloco é crucial.
-

8. Árvore B+ (B+ Tree)

- **Organização dos dados:** Variante da Árvore B onde os dados reais (ou ponteiros diretos para os registros) ficam armazenados exclusivamente nos nós folha, enquanto os nós internos armazenam apenas chaves de indexação/direcionamento.
 - **Propriedade mantida:** As folhas são interligadas sequencialmente por meio de ponteiros bidirecionais (formando uma lista encadeada nas folhas).
 - **Busca e Inserção:** A busca sempre desce até as folhas, independentemente de encontrar a chave no meio do caminho. A inserção divide nós internos e folhas conforme os limites de capacidade, mantendo os encadeamentos atualizados.
 - **Ajustes:** Divisões de nós e manutenção dos ponteiros sequenciais das folhas.
 - **Contexto de utilidade:** Sistemas de gerenciamento de banco de dados (SGBDs) modernos e indexação de arquivos para consultas por intervalo eficientes (*range queries*).
-

9. Heap (Árvore Heap)

- **Organização dos dados:** Árvore binária quase completa estruturada em vetor ou nós.
 - **Propriedade mantida:** Na *Max-Heap*, o valor de cada pai é maior ou igual ao de seus filhos; na *Min-Heap*, o valor do pai é menor ou igual ao dos filhos.
 - **Busca e Inserção:** A busca por um elemento arbitrário é lenta ($O(N)$), mas a busca pelo extremo (máximo ou mínimo) é $O(1)$ na raiz. A inserção coloca o novo elemento na última posição livre e aplica o processo de subida (*heapify-up* / *percolate-up*).
 - **Ajustes:** Reorganizações locais (*swaps*) baseadas na comparação com o pai para restaurar a propriedade de heap.
 - **Contexto de utilidade:** Implementação de filas de prioridade (*priority queues*) e algoritmos de ordenação (Heapsort).
-

10. Árvore Trie (Árvore de Prefixos)

- **Organização dos dados:** Árvore orientada a dados onde cada aresta representa um caractere e os caminhos a partir da raiz formam palavras.
- **Propriedade mantida:** Os nós filhos de um nó compartilham um prefixo comum em comum correspondente ao caminho percorrido até eles.
- **Busca e Inserção:** A busca percorre caractere por caractere descendo pelos ramos correspondentes ($O(M)$, onde M é o tamanho da chave). A inserção cria novos nós para caracteres que ainda não possuem um caminho mapeado.
- **Ajustes:** Remoção de nós órfãos caso palavras sejam deletadas e ramificações fiquem vazias.

- **Contexto de utilidade:** Sistemas de autocompletar (*autocomplete*), dicionários preditivos, corretores ortográficos e algoritmos de roteamento de IP.

Etapa 2 - Quadro comparativo

Estrutura	Organização dos dados	Regra ou propriedade	Operação ou ajuste	Aplicação
ABB	Nós ordenados binariamente por chave.	Subárvore esquerda menor; subárvore direita maior.	Busca comparativa e inserção na posição nula correspondente.	Tabelas de símbolos dinâmicas e conjuntos ordenados.
AVL	Árvore binária com controle de balanceamento estrito por fator de altura.	Fator de balanceamento de cada nó deve ser -1, 0 ou 1.	Rotações simples (LL, RR) e duplas (LR, RL) após desbalanceamentos.	Ambientes que exigem buscas rápidas e restritas sem degradação linear.
Rubro-negra	Árvore binária com atributos de cor (vermelho ou preto) associados aos nós.	Regras rígidas de alternância de cores e altura negra constante nos caminhos.	Recolorações de nós e rotações para restaurar as propriedades violadas.	Bibliotecas padrão de linguagens (<code>std::map</code> , <code>TreeMap</code>).
B	Estrutura de múltiplas vias (multivias) organizada em nós com capacidade para várias chaves.	Todas as folhas estão no mesmo nível; limites estritos de ocupação por nó.	Divisões (<i>splits</i>) de nós quando há transbordamento e fusões em remoções.	Sistemas de arquivos e indexação de bases de dados em disco.
B+	Multivias com separação entre índices internos e dados armazenados	Folhas encadeadas sequencialmente e em lista para percurso linear.	Divisões de nós e manutenção dos ponteiros de encadeamento nas folhas.	SGBDs relacionais para suporte a consultas por

Estrutura	Organização dos dados	Regra ou propriedade	Operação ou ajuste	Aplicação
	estritamente nas folhas.			intervalo eficientes.
Árvore Geral	Nós com capacidade de possuir um número ilimitado de filhos.	Relação estrita hierárquica pai-filho multivalorada.	Inserção direta de sub-ramos; reestruturação manual se necessário.	Sistemas de arquivos de SO e representação de documentos estruturados.
Árvore Binária	Nós com restrição estrita de no máximo dois filhos por nó.	Grau máximo de ramificação fixado em dois.	Inserção baseada em posições livres sequenciais ou específicas.	Árvores sintáticas de compiladores e expressões aritméticas.
Heap	Estrutura quase completa modelada tipicamente em vetores.	Pai sempre maior ou igual aos filhos (Max-Heap) ou menor/igual (Min-Heap).	<i>Heapify</i> (subida/descida de elementos) após inserção ou remoção da raiz.	Filas de prioridade e algoritmo Heapsort.
Trie	Organização baseada em caracteres por arestas ramificadas.	Caminhos compartilhados a partir da raiz baseados em prefixos comuns.	Inserção por adição de nós caractere a caractere.	Mecanismos de autocompletar e dicionários preditivos.

Etapa 3 - Identificação por analogias

1. **Situação:** Uma estante de números é reorganizada por rotações quando um lado fica alto demais em relação ao outro.
 - **Estrutura:** Árvore AVL.
 - **Justificativa técnica:** A propriedade de balanceamento restrito da AVL monitora a altura das subárvores e aplica rotações imediatas quando a diferença de altura excede o limite tolerado.

- **Limite da analogia:** Na prática da computação, a árvore não reorganiza livros fisicamente pelo peso visual, mas sim ponteiros de memória de nós lógicos baseados em fatores de balanceamento numéricos calculados.
2. **Situação:** Um catálogo guarda várias chaves por página; quando uma página fica cheia, ela é dividida.
- **Estrutura:** Árvore B.
 - **Justificativa técnica:** Na Árvore B, os dados são organizados em nós de capacidade múltipla (várias chaves por nó), e o transbordamento de capacidade aciona uma operação de divisão (*split*).
 - **Limite da analogia:** As páginas físicas de um livro real possuem espaço estático de caracteres e linhas, ao passo que os nós de uma Árvore B ajustam dinamicamente a alocação de ponteiros e chaves em blocos de memória ou disco.
3. **Situação:** Uma fila mantém a tarefa de maior prioridade no topo para retirá-la primeiro.
- **Estrutura:** Heap (Max-Heap ou Min-Heap).
 - **Justificativa técnica:** A propriedade fundamental da Heap garante que o elemento de maior (ou menor) prioridade ocupe sempre a raiz da estrutura, permitindo acesso imediato.
 - **Limite da analogia:** Uma fila comum do cotidiano é estritamente FIFO (First-In, First-Out) com base na ordem de chegada, enquanto a Heap prioriza os elementos com base em uma chave de prioridade intrínseca, ignorando a ordem temporal pura de inserção.
4. **Situação:** Um índice percorre letras sucessivas e compartilha o início das palavras de mesmo prefixo.
- **Estrutura:** Árvore Trie (Árvore de Prefixos).
 - **Justificativa técnica:** A Trie armazena strings de forma compacta explorando prefixos comuns compartilhados ao longo de caminhos direcionados por caracteres nas arestas.
 - **Limite da analogia:** Em um sumário ou índice remissivo tradicional de um livro físico, as palavras completas são listadas separadamente ou agrupadas de forma estática por letra inicial, sem a compactação de nós encadeados caractere a caractere de uma Trie.
5. **Situação:** Uma estrutura usa cores, recolorações e rotações para manter controlada a altura dos caminhos de busca.
- **Estrutura:** Árvore Rubro-Negra.

- **Justificativa técnica:** O uso de metadados de cores (vermelho e preto) associados a regras estritas de altura negra garante o balanceamento aproximado sem o custo de rotações excessivas da AVL.
 - **Limite da analogia:** Cores no mundo real carregam significados visuais ou semânticos (como alertas ou categorias), enquanto na árvore rubro-negra a cor é apenas um atributo booleano de controle lógico para balanceamento algorítmico.
6. **Situação:** Um índice conduz às folhas que contêm os registros, ligadas entre si para facilitar consultas por intervalo.
- **Estrutura:** Árvore B+.
 - **Justificativa técnica:** A separação entre chaves de roteamento nos nós internos e dados/ponteiros nas folhas, combinada com o encadeamento sequencial das folhas, caracteriza distintamente a Árvore B+.
 - **Limite da analogia:** Um índice remissivo de um livro aponta apenas para o número da página onde o termo ocorre, exigindo leitura sequencial manual das páginas, enquanto os ponteiros de uma B+ conectam diretamente blocos de dados contíguos na memória ou armazenamento secundário.
7. **Situação:** Numa coleção de números, cada nó direciona valores menores para a esquerda e maiores para a direita.
- **Estrutura:** Árvore Binária de Busca (ABB).
 - **Justificativa técnica:** A regra de ordenação estrita em que a subárvore esquerda contém apenas chaves inferiores e a direita chaves superiores define a propriedade estrutural da ABB.
 - **Limite da analogia:** Em uma classificação física de objetos por tamanho em caixas, a divisão é estática e definitiva, enquanto na ABB a inserção de novos elementos pode alterar dinamicamente a profundidade e a ramificação da árvore sem realocar toda a estrutura.
-